ACADEMIUM ENGINE PACKET

Section 1: Pipeline Architecture and Orchestration

Version: 2.0 **Status:** Active Specification **Role:** Master reference. Read this section first. All other sections operate within the architecture defined here. **Governs:** The complete Academium simulation engine — every input, every output, every section, every turn, in every case.

1.1 Engine Identity

The Academium engine is a **decision simulation engine that makes reasoning observable.**

This single statement governs every architectural decision in this packet. Every section, every rule, every constraint exists to serve it. When any ambiguity arises in implementation, return to this statement and ask: does this behavior make reasoning observable? Does it support a decision simulation? If the answer to either question is no, the behavior is wrong.

What the Engine Is

- A stateful environment where students make business decisions under realistic constraints
- A system that surfaces the consequences of those decisions in ways that reveal trade-offs
- A tool that makes the student's own reasoning visible through the experience of seeing what their choices produce
- An evidence generator that produces structured, auditable data about how students think

What the Engine Is Not

The engine is NOT	Why this boundary exists
A tutoring system	It does not teach content. It creates conditions in which reasoning is exercised.
A testing system	It does not grade performance. It records evidence of reasoning patterns.
A chatbot	It does not hold open-ended conversations. Every interaction is structured within a defined simulation framework.

The engine is NOT	Why this boundary exists
A game	It does not optimize for entertainment. It optimizes for learning fidelity and psychological safety.
An authority	It does not determine what is correct. There are no correct answers.

The Two Users

The engine serves two users with fundamentally different relationships to the system:

The Student interacts with the engine as a participant — making decisions, observing consequences, building understanding through experience. The student never sees the engine's mechanics. They see a simulation.

The Professor interacts with the engine as a configurator — selecting cases, defining parameters, setting KPIs, and choosing structural options. The professor defines the space within which the simulation operates. The engine operates within that space. The engine never overrides professor authority, and it never attempts to infer what a specific professor would consider the "right" answer.

1.2 Foundational Architecture Principles

Six principles govern the architecture of the entire engine pipeline. They are not design preferences — they are structural constraints. Any feature, optimization, or edge-case handler that violates any of these principles has introduced a defect, not an improvement.

Principle 1: Single Responsibility Per Section

Each section of the pipeline owns exactly one function. Section 2 extracts signals. Section 3 generates narratives. Section 4 computes metrics. Section 5 generates causal explanations. Section 6 assembles and validates. No section performs two distinct functions. If a section's scope expands to cover two functions, it must be split.

Principle 2: Stateful World, Stateless Sections

All simulation state lives in the session state object. Each pipeline section is a pure function: it receives inputs, produces outputs, and holds no state of its own. This means any section can be retried, replaced, or debugged independently without corrupting the simulation's memory. The session state is the single source of truth for everything that has happened in a simulation.

Principle 3: Parallel Where Possible, Sequential Where Necessary

Sections 3 (Consequence Generation) and 4 (KPI Computation) run in parallel. They receive the same inputs and operate independently. Neither determines the other's output. All other sections are sequential — each receives outputs from the section before it. The parallel architecture between Sections 3 and 4 is a deliberate design choice: if the narrative determined the metrics, the engine would be reverse-engineering a scorecard. If the metrics determined the narrative, the engine would be illustrating numbers. Both emerge from the same source — the student's signals and the case's directional logic — independently.

Principle 4: Validate Before Generate, Validate Before Deliver

Section 6 is the engine's final validation layer. Nothing reaches the student without passing through it. But validation also happens within each section — every section's output passes its own quality gates before being passed downstream. The pipeline has multiple validation layers, not one. This layered approach means that by the time Section 6 runs its cross-section consistency checks, most issues have already been caught and corrected upstream.

Principle 5: No Section Generates Content for Another

Sections 3, 4, and 5 generate content independently. Section 6 assembles it. No section reads another section's in-progress output and builds on it mid-generation. This prevents circular dependencies and ensures that each section's output is independently auditable.

Principle 6: Explainability Over Sophistication

Every output can be traced back through the pipeline to the student's input through an auditable chain. The chain is: Student input → Signal extraction (Section 2) → Content generation (Sections 3, 4, 5) → Assembly (Section 6) → Delivery. No step in this chain is a black box. If someone asks "why did this KPI move?", there is a traceable answer that runs from the student's specific words to the specific signal detected to the specific direction and tier assigned to the causal explanation generated.

1.3 Complete Pipeline Architecture

The Pipeline Map

```
═══════════════════════════════════════════════
PROFESSOR CONFIGURATION (pre-session)
  Case | Step count (4-10) | Active KPIs (3-5) |
  Language | Tradeoff config | Hint/Regen settings
═══════════════════════════════════════════════
```

```
                    ↓
          SECTION 7 — INPUT RECEPTION
          Receive input | Validate format |
          Timestamp | Initial classification
                    ↓
          SECTION 2 — SIGNAL EXTRACTION
          Classify PASS/NUDGE/BLOCK |
          Extract 5 signals | Compute RDS band |
          Map competency evidence
                    ↓
       ┌────────────┴────────────┐
       │  [PASS path only]       │
       ↓                         ↓
SECTION 3                  SECTION 4
CONSEQUENCE                KPI COMPUTATION
GENERATION                 Direction logic |
Narrative |                Tier assignment |
Compounding |              Accumulation model |
Tradeoff config            Anti-pattern checks
applied                    applied
       │                         │
       └────────────┬────────────┘
                    ↓
       SECTION 5 — CAUSAL EXPLANATION GENERATION
       One explanation per displayed KPI |
       Signal anchoring by RDS band |
       Three-part causal structure
                    ↓
       SECTION 6 — RESPONSE ASSEMBLY
       Input validation | Cross-section checks |
       Routing (PASS/NUDGE/BLOCK) |
       Response assembly by turn type |
       Quality gates | Graceful degradation
                    ↓
           ┌────────┴────────┐
           ↓                 ↓
 PRESENTATION LAYER      SESSION STATE UPDATE
 (UI rendering)          (enables compounding
                          in subsequent turns)
```

The Three Execution Paths

The pipeline executes differently depending on the student's input classification.

PASS Path — Full pipeline execution: Section 7 → Section 2 → [Section 3 + Section 4 in parallel] → Section 5 → Section 6 → Delivery + Session state update

NUDGE Path — Pipeline halts after Section 2: Section 7 → Section 2 (classification only) → Section 6 (NUDGE assembly) → Nudge callout delivered. Session state records NUDGE event. Turn counter does not advance.

BLOCK Path — Pipeline halts after Section 2: Section 7 → Section 2 (classification only) → Section 6 (BLOCK assembly) → Redirect message delivered. Session state records BLOCK event. Turn counter does not advance.

MCQ vs FR Execution

The pipeline also runs differently based on the decision format of the current turn.

MCQ Execution (always Turn 1, always PASS):

- Section 2 does not extract student-text signals — there is no free-response text
- Section 2 reads the pre-authored tradeoff signature of the selected option instead
- Sections 3 and 4 derive all content from the tradeoff signature, not from signal extraction
- Signal anchoring in Section 5 is not applicable
- RDS band is not computed — Section 3 uses the tradeoff signature's pre-authored richness level

FR Execution (all turns after Turn 1):

- Full signal extraction runs in Section 2
- RDS band governs richness across Sections 3, 4, and 5
- Signal anchoring applies at Engaged and Integrated bands

1.4 Section Dependency Map

Each section's inputs and outputs are precisely defined. No section may receive an input that is not on this map, and no section may produce an output that is not on this map.

Section	Receives From	Produces For
Section 7 (Input Reception)	Student, UI, Session state	Section 2
Section 2 (Signal Extraction)	Section 7, Case definition	Sections 3, 4, 5, 6, Session state
Section 3 (Consequence Generation)	Section 2, Session state, Case definition	Sections 5, 6
Section 4 (KPI Computation)	Section 2, Session state, Case definition	Sections 5, 6
Section 5 (Causal Explanations)	Sections 2, 3, 4	Section 6
Section 6 (Response Assembly)	Sections 2, 3, 4, 5, Session state, Case definition	Presentation Layer, Session state

Critical rule: No section reads from or writes to the session state mid-execution. Section state is read at the start of the turn (by Sections 2, 3, and 4 for context) and written at the end of the turn (by Section 6 after delivery). This prevents mid-turn state corruption.

1.5 The Five Behavioral Invariants

These five invariants govern all engine behavior without exception. They are implemented across specific sections of this packet. This table maps each invariant to its implementation.

Invariant	Definition	Implemented In
Mentorship Posture	The engine speaks as a calm, experienced colleague. It presents information neutrally, acknowledges complexity, and never evaluates the student's decision quality.	Section 3 (prohibited language, tone tests), Section 5 (prohibited inferences, signal anchoring constraints)
Safety Posture	The engine maintains psychological safety at all times. Hostile, profane, or off-topic inputs are redirected without judgment. The student is never shamed, lectured, or penalized.	Section 6 (BLOCK routing, redirect messages), Section 7 (input classification, profanity handling)
Realism Posture	Once a decision is submitted and consequences are delivered, the decision is locked. There is no undo. This mirrors real professional environments where decisions have lasting effects.	Section 6 (session state — decisions written as immutable records after delivery), Section 7 (submit action irreversibility)
Revision Posture	The engine never deletes, clears, or overwrites what the student wrote. On NUDGE or BLOCK, the student's text remains in the input field for editing.	Section 6 (NUDGE and BLOCK pathway — text preservation rule), Section 7
Tradeoff Posture	Every decision in the simulation involves genuine trade-offs when configured by the professor. The engine never presents artificially forced trade-offs, and never manufactures a consequence-free decision.	Section 3 (tradeoff configuration system), Section 4 (anti-pattern corrections, uniform direction prevention)

1.6 Professor Configuration Boundaries

The professor configures the space within which the engine operates. The engine operates within that space. These boundaries are absolute.

What the Professor Controls

Parameter	Options
Case	Which scenario the simulation uses
Step count	4–10 turns
Active KPIs	Any 3–5 of the 5 canonical KPIs

Parameter	Options
Tradeoff configuration	Specific text provided / enabled without text / not configured
Language	Spanish / English
Hint button	Enabled / disabled
Regeneration	Enabled / disabled

What the Engine Controls

The engine controls everything that happens within the configured space: signal extraction, classification, consequence generation, metric computation, causal explanation generation, response assembly, and session state management. The engine makes no decisions that belong to the professor, and the professor makes no decisions that belong to the engine.

What Neither Controls

- Whether a student's decision is correct. There is no correct decision.
- The student's experience of reasoning through the decision.
- The direction the student's thinking takes.

1.7 The Session State Object

The session state is the engine's memory. It is the only persistent data structure in the pipeline. Every section reads context from it; Section 6 writes to it after each turn. Without it, compounding logic, stakeholder continuity, KPI accumulation, and dashboard reporting are all impossible.

The session state is the implementation of the Stateful World, Stateless Sections principle. Sections do not hold memory — the state does.

What the Session State Contains

```
SESSION STATE

case_id: string
professor_config: {
  step_count: integer (4-10),
  active_kpis: array (3-5 KPI IDs),
  tradeoff_config: "specific_text" | "enabled" | "none",
```

```
  language: "es" | "en",
  hint_enabled: boolean,
  regeneration_enabled: boolean
}
session_status: "active" | "complete"
current_turn: integer
turns: [
  {
    turn_number: integer,
    turn_position: "first" | "intermediate" | "final",
    timestamp: ISO 8601,
    classification: "PASS" | "NUDGE" | "BLOCK",
    student_input: string,
    rds_band: "surface" | "engaged" | "integrated",
    signals_detected: { intent, justification, tradeoff_awareness,
                        stakeholder_awareness, ethical_awareness },
    kpi_movements: [{ kpi_id, direction, tier, displayed }],
    competency_evidence: { C1, C2, C3, C4, C5 },
    narrative_delivered: string,
    nudge_count: integer
  }
]
kpi_accumulation_state: {
  K1: { trajectory, consecutive_negative_turns,
        consecutive_positive_turns, last_tier },
  ...repeated for each active KPI
}
```

1.8 Canonical Terminology

The following terms are used with precise, consistent meanings throughout all sections of this packet. Where multiple terms exist for the same concept in the source material, this packet standardizes on one.

Canonical Term	Definition	Deprecated Synonyms
Turn	One complete decision cycle: prompt → student input → pipeline execution → response delivery	"Step," "Decision point," "Round"
Consequence	The narrative paragraph delivered after a student submits a PASS-level decision	"Outcome narrative," "Result" (reserved for Final Outcome)
Final Outcome	The trajectory summary delivered after the Final turn, before the Reflection prompt	"Final consequence," "End state"
Tradeoff Signature	The pre-authored metadata attached to each MCQ option encoding KPI direction and magnitude	"Option weights," "MC metadata"
RDS	Reasoning Depth Score — an internal pipeline variable (1–15) that governs consequence richness and explanation depth. Never displayed.	"Depth score," "Reasoning score"
Signal	One of five extractable reasoning markers in a student's FR response: Intent, Justification, Tradeoff Awareness, Stakeholder Awareness, Ethical Awareness	"Indicator," "Reasoning marker"
Competency	One of five Academium competency dimensions (C1–C5) mapped from signal combinations	"Skill," "Learning outcome"
Session	One complete simulation run by one student in one case	"Run," "Instance," "Attempt"
Turn position	Whether a turn is the First, an Intermediate, or the Final turn in the session — positional, not absolute	"Turn number" (used only for state tracking, not logic)

1.9 What "Bulletproof" Means

The engine is operating correctly — bulletproof — when all of the following conditions are simultaneously true:

1. **Traceability:** Every element of every student-facing response can be traced through the pipeline to the student's specific input via an auditable chain.

2. **Invariant compliance:** No response violates any of the five behavioral invariants under any input condition — including adversarial inputs, minimal inputs, hostile inputs, and off-topic inputs.
3. **Prohibited language absence:** No prohibited term or construction (Sections 3.7, 5.8) appears in any student-facing output in any session.
4. **KPI accountability:** Every KPI movement displayed to the student has a causal explanation accessible via the "Why did this change?" affordance, and that explanation correctly references the student's decision.
5. **Narrative coherence:** Every consequence narrative maintains continuity with all prior turns in the session — stakeholders are consistent, metrics accumulate correctly, and the organizational story does not contradict itself.
6. **Classification accuracy:** PASS, NUDGE, and BLOCK are applied correctly — no legitimate decision is blocked, no empty or hostile input is processed as a PASS, and NUDGE is never used to punish brevity.
7. **Graceful failure:** When any section fails, the engine degrades gracefully — the student sees a neutral notification, their progress is preserved, and no partial or malformed response is delivered.

1.10 How to Read This Packet

This packet is structured to be read sequentially by someone implementing or fine-tuning the engine for the first time, and referenced non-sequentially by someone debugging or updating a specific section.

Sequential reading order: Section 1 → Section 7 → Section 2 → Sections 3 and 4 → Section 5 → Section 6

Reference order for debugging:

- Input classification issues → Section 7
- Signal extraction issues → Section 2
- Consequence tone or content issues → Section 3
- KPI direction or tier issues → Section 4
- Causal explanation issues → Section 5
- Response structure or consistency issues → Section 6
- Architecture-level questions → Section 1

Update protocol: Changes to any section must be reviewed against Section 1 to confirm they do not violate any foundational architecture principle or behavioral invariant. Changes that affect the session state schema must be propagated to all sections that read from or write to session state (currently: Sections 2, 3, 4, and 6). Changes to prohibited language lists must be propagated to Sections 3, 5, and 6 simultaneously.

Section 1 is the governing document for the entire engine packet. All other sections operate within the architecture, principles, and constraints defined here. In any conflict between this section and another section of the packet, Section 1 takes precedence.